GESTION ÉCOLE

Rapport technique

Application web de gestion d’un établissement scolaire

GROUPE 6

DAMASSOH Armel · AHOTON Lydia · AHOUANSSOU Fany
HOUKADJI Maxime · AIMONTCHE Géraud

Frontend React/Vite · API Laravel · MySQL

Dépôt : github.com/Armel1617/gestion-ecole

Sommaire

Une solution complète séparée en une interface moderne et une API métier sécurisée.

N°	Partie
01	Vision et architecture
02	Technologies
03	Fonctionnalités frontend
04	Fonctionnalités backend
05	Rôles et permissions
06	Installation et utilisation
07	Validation et recommandations

Résumé exécutif

Gestion École centralise le suivi des étudiants, des formations, des inscriptions, des paiements, des notes, des présences, des documents et des notifications. Le frontend React communique avec une API Laravel versionnée et protégée par des jetons Sanctum. L'interface adapte les accès au rôle de chaque utilisateur.

Dépôt GitHub : https://github.com/Armel1617/gestion-ecole

1. Vision et architecture

L'application adopte une architecture client–API. Le frontend est chargé par le navigateur et affiche les écrans métier. Le backend expose les données et applique les validations, les permissions et les règles de sécurité. MySQL conserve les informations de l'établissement.

Couche	Responsabilité	Emplacement
Frontend	Navigation, formulaires, tableaux, graphiques, notifications et expérience utilisateur	Racine du dépôt
Backend	Authentification, API REST, rôles, validations, règles métier et audit	backend/
Données	Utilisateurs, étudiants, inscriptions, paiements et suivi scolaire	MySQL

Flux principal : connexion → jeton Sanctum → requête Axios vers /api/v1 → middleware d'authentification et de rôle → contrôleur Laravel → réponse JSON paginée → affichage React.

2. Technologies et composants

Frontend

- React 19 et Vite pour l'interface et le build.
- React Router pour les routes publiques, privées et par rôle.
- Axios pour les appels HTTP avec ajout automatique du jeton Bearer.

- Recharts pour les graphiques du tableau de bord.
- Lucide React pour les icônes et Oxlint pour le contrôle du code.

Backend

- PHP 8.3+ et Laravel 13.
- Laravel Sanctum pour l'authentification par jeton.
- Eloquent ORM et migrations pour la base de données.
- PHPUnit pour les tests, Pint pour le formatage et queues Laravel pour les notifications.

Sécurité : les fichiers .env, les dépendances, les logs et les fichiers générés sont exclus du dépôt GitHub.

3. Fonctionnalités frontend

Authentification et navigation

- Connexion par e-mail et mot de passe.
- Création publique d'un compte étudiant.
- Routes protégées et écran d'accès interdit.
- Menu latéral escamotable avec bouton hamburger.
- Navigation filtrée selon le rôle.

Tableau de bord

- Statistiques étudiants, formations et inscriptions.
- Revenus estimés, inscriptions actives et évolution mensuelle.
- Graphiques et cartes statistiques.
- États squelette et sécurisation contre les écrans blancs.

Étudiants, formations et inscriptions

- Listes paginées, recherche et filtre par formation.
- Création, modification et suppression selon les permissions.
- Création d'une inscription avec date et statut.
- Messages d'erreur explicites lorsque l'API ou MySQL est indisponible.

Paiements, notes et présences

- Création de paiements reliés à une inscription.
- Formulaire de note : matière, valeur, barème, coefficient et commentaire.
- Formulaire de présence : date, statut, retard et motif.
- Validation des erreurs renvoyées par Laravel.

Notifications et performance

- Cloche visible dans la barre supérieure.
- Compteur des notifications non lues et bouton « Tout lire ».
- Consultation rapide sans quitter la page.
- Chargement différé des pages avec React.lazy et bundle initial réduit.

4. Fonctionnalités backend

Authentification

- POST /api/connexion : connexion et création d'un jeton.
- POST /api/creer-compte : création exclusive d'un étudiant.
- GET /api/v1/me et POST /api/v1/deconnexion.
- PUT /api/v1/mot-de-passe : changement du mot de passe.
- Limitation des tentatives de connexion et réponses JSON uniformes.

Ressources API versionnées

Ressource	Endpoint principal	Fonctions
Étudiants	/api/v1/etudiants	Lister, créer, modifier, supprimer
Formations	/api/v1/formations	Lister, créer, modifier, supprimer
Inscriptions	/api/v1/inscriptions	Créer, lister, modifier, supprimer
Paiements	/api/v1/paiements	Lister et enregistrer
Notes	/api/v1/notes	Lister et enregistrer
Présences	/api/v1/presences	Lister et enregistrer
Documents	/api/v1/documents	Lister, déposer et télécharger
Notifications	/api/v1/notifications	Consulter, lire une ou toutes
Utilisateurs	/api/v1/utilisateurs	Créer et administrer selon le rôle

Les listes sont paginées par Laravel. Le paramètre per_page accepte une valeur de 1 à 100.

5. Rôles et permissions

Les rôles sont administrateur, direction, secretaire, enseignant et etudiant. Les permissions sont appliquées côté frontend pour l'expérience utilisateur et côté backend pour la sécurité réelle.

Action	Admin	Direction	Secrétaire	Enseignant	Étudiant
Tableau de bord	Oui	Oui	Oui	Oui	Non
Étudiants	Oui	Oui	Oui	Oui	Non
Paiements	Oui	Oui	Oui	Non	Personnel
Notes / présences	Oui	Oui	Non	Oui	Personnel
Créer un utilisateur	Tous rôles	Rôles inférieurs	Non	Non	Non
Modifier un rôle	Oui	Non	Non	Non	Non
Changer son mot de passe	Oui	Oui	Oui	Oui	Oui

La direction ne peut pas créer un administrateur ni un autre compte de direction. L'administrateur peut modifier le rôle et le mot de passe des utilisateurs.

6. Règles métier et sécurité

- Une inscription utilise automatiquement l'année académique active si aucune année n'est fournie.
- Une note ne peut pas dépasser son barème et les notes sont normalisées sur 20 dans le bulletin.
- Une présence est unique pour une inscription et une date.
- Les documents sont conservés sur le disque privé Laravel.
- Les mutations API sont enregistrées dans audit_logs avec les secrets expurgés.
- Les paiements peuvent générer des notifications pour l'administration et la direction.
- Les secrets de configuration restent dans .env et ne sont jamais publiés.

7. Installation et démonstration

Prérequis

- Git, Node.js/npm, PHP 8.3+, Composer, MySQL et Laragon ou environnement équivalent.
- Apache n'est pas obligatoire si le backend est lancé avec php artisan serve. MySQL doit être démarré.

Installation

```
git clone https://github.com/Armel1617/gestion-ecole.git
cd gestion-ecole

cd backend
composer install
copy .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve
```

Dans un second terminal :

```
cd gestion-ecole
npm install
npm run dev
```

Le frontend utilise l'API http://127.0.0.1:8000/api/v1.

Compte de démonstration

E-mail : **admin@ecole.local**
Rôle : **administrateur**
Mot de passe : temporaire, transmis séparément et à changer après la première connexion.

8. Validation, limites et conclusion

Vérifications réalisées

- npm run build : compilation frontend réussie.
- npm run lint : analyse exécutée avec des avertissements non bloquants.
- php -l routes/api.php : syntaxe PHP correcte.
- php artisan route:list : routes versionnées confirmées.
- Vérification des routes Notes, Présences, Notifications et Utilisateurs.
- Vérification de l’exclusion des .env, vendor, node_modules et logs.

Points à surveiller

- Démarrer MySQL avant la consultation des données.
- Changer le mot de passe administrateur temporaire.
- En production, pointer le serveur web vers backend/public/.
- Configurer un worker Laravel pour les notifications en queue.

Conclusion

Le groupe 6 a réalisé une solution complète de gestion scolaire avec une interface React responsive et une API Laravel sécurisée. Le projet couvre l’authentification, les rôles, le suivi pédagogique et financier, les notifications et les règles métier principales. Le code est disponible sur GitHub, avec le frontend à la racine et le backend dans backend/.

Document préparé pour la présentation du projet — Groupe 6